



Problem A. Permatomo Tasters (Easy Version)

Source file name:	Permatomo.c, Permatomo.cpp, Permatomo.java, Permatomo.py
Input:	Standard
Output:	Standard
Time / Memory limit:	2/2/2 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	Nicolas Eduardo Pardo Arias - UNAL Bogotá, Colombia

This is the easy version of the problem. Note that the only difference in the problems are the constraints over N , K and T .

Mari and Klaus are a beautiful couple who love mathematics. They also share a very particular passion: they are **Permatomo tasters**. The Permatomo is an abstract object that represents the combinatorial essence of couples, and Mari and Klaus enjoy experimenting with it in creative ways. Today, they are looking to obtain the **k -th best couple of the Permatomo**.

Formally, given an array of size N , Mari and Klaus want to compute the sum of **all possible pairs** (i, j) where $1 \leq i, j \leq N$. For each pair (i, j) they calculate: $s_k = a_i + a_j$ where k corresponds to the number of the couple created. In total, there are exactly N^2 possible couples.

Then, Mari and Klaus ask themselves: What is the **k -th best couple of the Permatomo**? For Permatomo tasters, this means finding the **k -th smallest sum** among all possible sums s_k .

Input

The input contains several test cases.

The first line contains an integer T ($1 \leq T \leq 20$) the number of test cases.

Each test case has the following format:

- A line with two integers N and K ($1 \leq N \leq 500$, $1 \leq K \leq N^2$).
- A line with N integers $a_1, a_2, \dots$ ($-10^9 \leq a_i \leq 10^9$).

Output

For each test case, print a single integer: the **k -th best couple of the Permatomo**, that is, the **k -th smallest sum** among all the possible sums of couples.

Example

Input	Output
2	6
3 4	1
1 3 5	
2 2	
-1 2	

Use fast I/O methods



Problem B. Slanting the board

Source file name: Slanting.c, Slanting.cpp, Slanting.java, Slanting.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Jherson Medina - UNAL Bogotá, Colombia

We have a maze with T ($1 \leq T \leq 3$) spheric balls inside. We want to achieve a certain state of the maze by only slanting it to any of the four cardinal directions. So, for example, if we slant the maze to the north, then the spheric balls would move up the maze until they hit a wall, hit another ball, or hit the border of the maze. Given the initial and final states of the maze, say if we can achieve the final state only by slanting the maze to the four cardinal directions.

Input

The first line contains N and M ($1 \leq N \cdot M \leq 60$) — the number of rows and columns of the maze, respectively.

The next N lines each contain a string of M characters detailing the initial state of the maze. The character '.' represents a free space in the maze, 'X' represents a spheric ball, and '#' represents a wall.

The last N lines represent the final state of the maze, following the same convention as with the initial state.

Output

Print "YES" if we can achieve the final state of the maze by slanting the maze with the initial state to the four cardinal directions; otherwise, print "NO".



Example

Input	Output
3 3X. X.. ...	YES
3 3 ... X..X. ...	NO
5 5 ..#.. .X..# .#... ...#. .X... ..#.. .X..# .#.X. ...#.	YES

Use fast I/O methods

Problem C. Aboreo Warehouse

Source file name:	Aboreo.c, Aboreo.cpp, Aboreo.java, Aboreo.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	Miguel Angel Saez - UNAL Bogotá, Colombia

You are responsible for a company that produces and distributes Aboreo, an alcoholic beverage valued for the variety in its oplo degree, a component that provides sweetness. Using specialized rays, it is possible to precisely modify the oplo degree of stored bottles. You must develop a system to simulate the state of a warehouse where Aboreo bottles are stored and modified. Initially, the warehouse is empty. A total of q operations will be performed on the warehouse, each in one of the following forms:

1. **'1 x'** (Add bottle): A new bottle with an oplo degree of x ($1 \leq x \leq 10^8$) is added to the warehouse.
2. **'2 x'** (Increase oplo): The oplo degree of each bottle in the warehouse increases by x ($-10^8 \leq x \leq 10^8$). If the oplo degree of the j -th bottle was v_j before the operation, it becomes $v_j + x$ afterward.
3. **'3 p q'** (Multiply oplo): The oplo degree of each bottle is multiplied by $\frac{p}{q}$ ($-10^8 \leq p, q \leq 10^8$, $q \neq 0$, $p \neq 0$). If the oplo degree of the j -th bottle was v_j before the operation, it becomes $v_j \cdot \frac{p}{q}$ afterward.
4. **'4 j'** (Query degree): Print the oplo degree of the bottle added in operation j ($1 \leq j \leq i$), considering all operations performed up to that point.

Notes:

The oplo degree may become extremely large, small, negative, or non-integer after operations. For queries (type 4), it is guaranteed that the oplo degree of the queried bottle is an integer $0 \leq v \leq 10^8$, avoiding precision issues.

Implement an algorithm that performs this simulation efficiently.

Input

The input begins with an integer q indicating the number of operations to perform ($2 \leq q \leq 10^5$). This is followed by q lines, each describing an operation in one of the following formats:

- **'1 x'**: Add a new bottle with oplo degree x ($1 \leq x \leq 10^8$).
- **'2 x'**: Increase the oplo degree of all bottles by x ($-10^8 \leq x \leq 10^8$).
- **'3 p q'**: Multiply the oplo degree of all bottles by $\frac{p}{q}$ ($-10^8 \leq p, q \leq 10^8$, $q \neq 0$, $p \neq 0$).
- **'4 j'**: Query the oplo degree of the bottle added in operation j ($1 \leq j \leq i$, where i is the current operation number).

Additional constraints:

There will be at least one operation of type 4 (query). For type 4 operations, the oplo degree of the queried bottle is guaranteed to be an integer $0 \leq v \leq 10^8$. Additionally, operation j is always of type 1 (add bottle).

Output

For each operation of type **'4 j'**, print a single line containing an integer representing the oplo degree of the bottle added in operation j , after applying all operations up to that point.



Example

Input	Output
7 1 3 3 1 2 1 1 2 1 3 4 1 4 1 4 3	10 8
4 1 1 2 -6 3 3 -5 4 1	3

Explanation

In the first example, the following operations are performed in this order:

- A bottle arrives with an oplo degree of 3.
- The oplo degree of all bottles is multiplied by $\frac{1}{2}$. The bottle added in the first operation now has an oplo degree of $\frac{3}{2}$.
- A bottle arrives with an oplo degree of 1.
- The oplo degree of all bottles is increased by 1.
- The oplo degree of all bottles is multiplied by 4.
- Finally, the oplo degree of the bottle added in the first operation and the bottle added in the third operation must be printed.

Problem D. How many trujos?

Source file name: Trujos.c, Trujos.cpp, Trujos.java, Trujos.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Oscar Fabián Sierra - UNAL Bogotá, Colombia

A mapping company is testing a new signal simulation system. They have placed N signal towers at different positions on a flat map. Each tower is located at coordinates (x_i, y_i) , where $1 \leq x_i, y_i \leq 2 \cdot 10^5$. All y_i values are distinct and all x_i values are distinct as well, additionally, there are no three colinear points.

To analyze signal interference, the company draws a virtual line between every pair of towers. These lines are extended downward, and the engineers are interested in how these lines affect positions on a distant horizontal scan line at $y = -10^{18}$.

They define a function $f(x)$ for real numbers x , which represents interference at horizontal position x on the scan line. Initially, $f(x) = 0$ for all x .

Then, for every pair of towers $i \leq j$, the following process is applied:

- Consider the directed line from point i to point j .
- For a fixed real number x , consider the point $(x, -10^{18})$.
 - If this point is on or to the left of the line (remember that the line is directed from point i to point j), then update $f(x) := f(x) \oplus a$.
 - If it is to the right of the line, then update $f(x) := f(x) \oplus b$.

Here, $\oplus$ denotes the bitwise XOR operation.

After processing all pairs, the function $f(x)$ becomes piecewise constant: the real line is divided into a number of intervals where $f(x)$ has the same value. We define a **Trujo** as any such interval where $f(x) = 0$. Your task is to count how many Trujos appear on the real number line.

Input

The first line contains three integers N, a , and b ($2 \leq N \leq 2 \cdot 10^5, 0 \leq a, b \leq 10^9$).

The next N lines each contain two integers x_i, y_i ($1 \leq x_i, y_i \leq 4 \cdot 10^5$). All y_i values are distinct and all x_i values are distinct, additionally, there are no three colinear points.

Output

Print one integer - the number of **Trujos**, i.e., the number of intervals on the real number line where $f(x) = 0$.

Example

Input	Output
4 7 12 1 7 7 5 3 10 4 9	4



Problem E. ¡¡Telo, en ti erro!!

Source file name:	Telo.c, Telo.cpp, Telo.java, Telo.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	Juan David Martínez Herrera - UNAL Bogotá, Colombia

For many years, the kingdoms of Karsmir, ruled by King Telo, and Elyria, ruled by Queen Sofia, lived in perfect harmony. Their alliance was not only political; it was also the reflection of a deep love that had united their hearts since youth.

They shared victories, developed magical defense technology, and dreamed of building a peaceful world. However, this union ended suddenly when Sofia discovered that Telo had been secretly communicating with an old love from the northern lands. Telo claimed it was just a diplomatic exchange, but Sofia, feeling betrayed, decided to break the relationship—and the alliance. Now, both kingdoms are at war, and the situation is bleak.

In the kingdom of Karsmir, there are n underground bunkers placed in a row, numbered from 1 to n , from left to right. Each bunker has a depth d_i and an energy cost e_i .

Because of the air attacks ordered by Sofia, it is necessary to select the largest possible subset of bunkers that can be connected using magical communication without interference. To make this work, the following condition must be satisfied:

- **Close Energy Optimization**

A bunker i can only be selected if the first bunker to its right with a lower depth ($d_j < d_i$) also has a lower energy cost ($e_j < e_i$).

Formally, let j be the smallest index such that $j > i$ and $d_j < d_i$. Then, bunker i can be selected only if $e_i > e_j$.

If there is no bunker to the right of i with lower depth, then i can be selected freely.

(Bunkers closer to the surface interfere with the signal if they consume more energy than the deeper bunker trying to communicate)

Your task is to print the longest possible subsequence of bunkers that can be created following this condition.

Input

The first line contains an integer n ($3 \leq n \leq 10^6$) — the number of bunkers in Telo's kingdom.

The second line contains n space-separated integers $d_1, d_2, \dots, d_n$ ($100 \leq d_i \leq 10^9$) — representing the depth of each bunker. **All depths are distinct.**

The third line contains n space-separated integers $e_1, e_2, \dots, e_n$ ($1 \leq e_i \leq 10^9$) — representing the energy cost of each bunker.

Output

Output an integer k ($1 \leq k \leq n$) — the size of the longest subsequence of bunkers that can be selected following the above conditions.

Then, on the next line, output k integers $a_1, a_2, \dots, a_k$ ($1 \leq a_i < a_{i+1} \leq n$) — the indices of the bunkers that belong to the longest valid subsequence.



Example

Input	Output
6 120 110 130 111 105 140 15 8 30 10 30 2	4 1 3 5 6



Problem F. What is Obo?

Source file name:	Obo.c, Obo.cpp, Obo.java, Obo.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	Jherson Medina - UNAL Bogotá, Colombia

The eccentric mathematician and dimensional builder Mr. Padalustro has spent his life studying geometric shapes in higher dimensions. His latest obsession is a structure he calls an **Obo**.

An **Obo** is an n -dimensional rectangular box. Formally, it has $n \geq 1$ sides with positive integer lengths $(l_1, l_2, l_3, \dots, l_n)$, and its area is calculated by multiplying all the lengths:

$$\text{Area}_{\text{Obo}} = l_1 \cdot l_2 \cdot l_3 \cdot \dots \cdot l_n$$

Mr. Padalustro wants to build an **Obo** with area exactly equal to a given positive integer A . However, building an Obo is not easy: each side must be a positive integer greater than 1, and no side can be longer than X , because his tools cannot create sides longer than that.

He is also obsessed with dimensions: he wants to build an **Obo** with the **maximum number of dimensions possible** (i.e., the largest possible value of n such that the product of the side lengths is exactly A , and every side is less than or equal to X).

Your task is to help him determine the maximum number of dimensions such an Obo can have. If it is not possible to build any Obo with area exactly equal to A and side lengths not greater than X , you must print a special message.

Input

The input consists of a single line with two integers:

- A ($2 \leq A \leq 2 \cdot 10^7$) the desired area of the Obo.
- X ($2 \leq X < A$) the maximum allowed length for any side of the Obo.

Output

If it is possible, print a single integer n the number of sides of the Obo.

If it is not possible, print: “**what is Obo?**” without quotes.

Example

Input	Output
2520 8	7
30 4	what is Obo?



Problem G. S-parkour

Source file name: Sparkour.c, Sparkour.cpp, Sparkour.java, Sparkour.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Roger David Trujillo - UNAL Bogotá, Colombia

After losing the ICPC (International Collegiate Parkour Contest), you decided to participate in a parkour competition with different rules. You have traveled to Silvestre-land, where a new parkour competition will take place.

Similar to Diomedes-land, in Silvestre-land, there are N buildings with positive heights $h_1, \dots, h_N$, it is guaranteed that all building heights are distinct.

Each round of the competition will face two competitors, and it will take place from building i to building j , where $i \leq j$. One competitor will jump between buildings from i to j , while the other will jump from j to i .

When a competitor jumps between buildings, they may have to ascend or descend. If the height of the next building is greater than the current one, they must ascend; otherwise, they must descend. For example, if the competitor passes through buildings with heights 1, 7, 2, they first ascend and then descend, which we can represent with the sequence UD (Up, Down).

For a competition between buildings (i, j) to be fair, the number of ascents from i to j must be equal to the number of descents from j to i .

- For example, if the buildings between i and j have heights 1, 7, 2, the competition is fair because, from left to right, the sequence of ascents and descents is UD (one ascent and one descent), and from right to left, it is also UD (one ascent and one descent).
- However, if the buildings between i and j have heights 3, 2, 1, the competition would not be fair because the sequence from left to right is DD (no ascents and two descents), but from right to left, it is UU (two ascents and no descents).
- Finally, if the buildings between i and j have heights 3, 4, 2, 1, 5, the competition would be fair because the sequence from left to right is UDDU (two ascents and two descents), and from right to left, it is DUUD (two ascents and two descents).

To prepare for the competition, you want to know how many pairs (i, j) with $i \leq j$ result in a fair competition.

Input

The input consists of an integer $1 \leq N \leq 10^6$ representing the number of buildings in Silvestre-land, followed by N integers representing the heights of the buildings $h_1, \dots, h_N$. $1 \leq h_i \leq 10^9$ for every i .

Output

Print the number of pairs (i, j) such that the competition between buildings from i to j is a fair competition.

Example

Input	Output
6 6 1 4 9 7 3	10



Problem H. What is Tacafroto?

Source file name:	Tacafroto.c, Tacafroto.cpp, Tacafroto.java, Tacafroto.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	Oscar Fabián Sierra - UNAL Bogotá, Colombia

A group of friends is trapped inside an abandoned building that has been converted into a giant escape room. The building's layout is modeled as a simple, connected, undirected graph where each node represents a room and each edge represents a corridor connecting two rooms.

One of the friends -Mr. Tacafroto- managed to reach the control room. In this room, Tacafroto has full access to the building's lighting system and blueprints. Each corridor is equipped with its own light that can shine in a fixed color. However, the building is initially in complete darkness: all corridor lights are turned off. Tacafroto can activate the lights, but with one important constraint: at any given moment, he may turn on the lights of only one chosen color.

Before issuing any instructions, Tacafroto is allowed to set up the building's corridors as follows:

Assignment of Colors: Each corridor (i.e., every existing undirected edge) is assigned a color from the set $\{1, 2, 3\}$. That is, every corridor has its dedicated light of a specific color.

Optional Passage: He may also build at most one new one-way passage (a directed edge) connecting two rooms that are not already connected. This new passage is assigned a color (from 1 to 3) just like the other corridors.

The remaining friends are stuck in different rooms. Tacafroto does not know exactly how many friends there are, nor in which rooms they are located. To gather them, Tacafroto can issue a sequence of instructions. In each instruction he selects a color $c \in \{1, 2, 3\}$ and performs the following steps:

All corridor lights are turned off first.

Then, the lights for all corridors whose assigned color is c (including the light of the extra one-way passage, if one was built and have color c) are turned on.

Immediately after, every friend (regardless of their room) makes a move:

If there is at least one corridor from their current room whose light is on (i.e., whose assigned color is c), the friend must choose one such corridor and move to the adjacent room.

If there are multiple illuminated options, the friend may pick any of them arbitrarily. Tacafroto does not control these choices and has no knowledge of the friends movements.

If no corridor from their room is lit, the friend remains in place.

The goal is to design the initial color assignments (and optionally, add the directed passage) along with a sequence of at most 205 instructions —each instruction simply a color— to guarantee that, regardless of how many friends there are or where they start, all friends eventually meet in the same room.

Input

The first line contains two integers n and m — the number of rooms and the number of corridors in the building, respectively. $2 \leq n \leq 200$, $1 \leq m \leq \frac{n(n-1)}{2}$

Each of the next m lines contains two integers u and v ($1 \leq u, v \leq n$, $u \neq v$), indicating that there is a corridor connecting room u and room v .

It is guaranteed that:

- The graph is **connected**.

- There are **no multiple corridors** between the same pair of rooms.
- There are **no loops**, i.e., $u \neq v$.

Output

If it is impossible to guarantee that all friends will meet in the same room within at most 205 instructions, print “NO” in a single line (without quotes).

Otherwise, output the following:

- The first line should contain the string “YES” (without quotes).
- The second line should contain three integers u , v , and c ($1 \leq u, v \leq n$, $1 \leq c \leq 3$), representing a new **directed edge** from room u to room v with color c .
If you choose not to add a new edge, print “-1 -1 -1” (without quotes).
- The next m lines should each contain an integer $c_i \in \{1, 2, 3\}$, representing the color assigned to the i -th undirected corridor (in the same order as given in the input).
- The last line should contain a string of length at most 205, consisting only of the digits 1, 2, and 3. This string represents the sequence of instructions that Tacafroto gives to the friends. Each character corresponds to one move: the number indicates the color whose lights are turned on in that step.

Your output must guarantee that, no matter how many friends there are and in which rooms they start, all of them will eventually meet in the same room after following the given sequence of instructions.

Example

Input	Output
3 2 1 2 2 3	YES 2 3 1 2 3 121



Problem I. Permatomo Tasters (Hard Version)

Source file name:	Permatomo2.c, Permatomo2.cpp, Permatomo2.java, Permatomo2.py
Input:	Standard
Output:	Standard
Time / Memory limit:	2/2/2 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	Nicolas Eduardo Pardo Arias - UNAL Bogotá, Colombia

This is the hard version of the problem. Note that the only difference in the problems are the constraints over N , K and T .

Mari and Klaus are a beautiful couple who love mathematics. They also share a very particular passion: they are **Permatomo tasters**. The Permatomo is an abstract object that represents the combinatorial essence of couples, and Mari and Klaus enjoy experimenting with it in creative ways. Today, they are looking to obtain the k -th best couple of the Permatomo.

Formally, given an array of size N . Mari and Klaus want to compute the sum of **all possible pairs** (i, j) where $1 \leq i, j \leq N$. For each such pair (i, j) they calculate: $s_k = a_i + a_j$, where k corresponds to the number of the couple created. In total, there are exactly N^2 possible couples.

Then, Mari and Klaus ask themselves: What is the k -th best couple of the Permatomo? For Permatomo tasters, this means finding the k -th smallest sum among all possible sums s_k .

Input

The input contains several test cases.

The first line contains an integer T ($1 \leq T \leq 3$), the number of test cases.

Each test case has the following format:

- A line with two integers N and K ($1 \leq N \leq 100000, 1 \leq K \leq N^2$).
- A line with N integers $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$).

Output

For each test case, print a single integer: the k -th best couple of the Permatomo, that is, the k -th smallest sum among all the possible sums of couples.

Example

Input	Output
2	6
3 4	1
1 3 5	
2 2	
-1 2	

Problem J. Sergeant Camelas's Keywords

Source file name: Sergeant.c, Sergeant.cpp, Sergeant.java, Sergeant.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Sebastian Chaves - UNAL Bogotá, Colombia

In the bustling city of Chorzolandia, Sergeant Camelas, a sharp and experienced traffic officer, has just pulled over a driver for a violation. The sergeant is well aware that in this city, some drivers attempt to “negotiate” their fines through subtle, often indirect means. To effectively identify these illicit attempts, the sergeant has meticulously memorized five secret keywords. He understands that if these five words are spoken consecutively by a driver, forming any one of their 5 possible ordered arrangements, it constitutes a clear signal of attempted bribery, leading to a significantly escalated fine.

The driver, desperate to avoid the standard traffic fine and completely unaware of the sergeant's unique detection method, decides to launch into a long, rambling statement. Their hope is to “smooth over” the situation by trying out various combinations of words, including, unknowingly, permutations of the sergeant's secret keywords.

Your task is to analyze the driver's complete sentence. Specifically, you must determine if any of the 5! permutations of the five secret keywords appears as a contiguous substring within the phrase the driver says to the sergeant. If such a specific keyword sequence is detected, indicating a negotiation attempt, the fine will be increased. Otherwise, the original fine remains in effect.

Input

The input will consist of two lines:

- A single line containing the complete phrase (S) the driver says to the sergeant without spaces. The length of S will be between 1 and 10^4 characters. It will contain only english lowercase letters.
- A single line containing the five keywords, separated by a single space. Each keyword will consist only of lowercase letters and have a length between 1 and 10 characters.

Output

Print “**Nooo, la polizzia**” if any of the 5 permutations of the keywords are found as a contiguous substring in the driver's phrase, indicating that the fine will be higher. Otherwise, print “**Sargento Camelas, Gracias!**”, meaning the fine remains the same.

Example

Input
senoragentetengocienmilrazonesparasolucionaresto razones para solucionar mil cien
Output
Nooo, la polizzia
Input
senoragentetengocienmilrazonesparasolucionaresto razones para solucionar mil diez
Output
Sargento Camelas, Gracias!



Input
senoragente hay cien mil razones para solucionar esto agente cien mil razones solucionar
Output
Sargento Camelas, Gracias!

Problem K. Timulo and Candies

Source file name: Timulo.c, Timulo.cpp, Timulo.java, Timulo.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Miguel Angel Saez - UNAL Bogotá, Colombia

There are n candies in a row, numbered from 1 to n . The i -th of them has a **repose** v_i and a **color** c_i .

Timulo wants to choose a **subsequence** of candies, such that adjacent candies in the chosen subsequence have **different colors**.

Formally, if the indices of the candies that Timulo chose were $i_1 < i_2 < \dots < i_k$, then $c_{i_j} \neq c_{i_{j+1}}$ for all $1 \leq j \leq k - 1$.

Timulo wants to maximize the **sum of the repose** from the subsequence. Can you help him find the **maximum possible sum**?

A **subsequence** is a sequence that can be derived from the given sequence by deleting zero or more elements without changing the order of the remaining elements.

Input

The first line contains an integer n ($1 \leq n \leq 2 \cdot 10^5$): The number of candies.

The second line contains n integers $v_1, v_2, \dots, v_n$ ($0 \leq v_i \leq 10^9$), denoting the **repose** of the candies.

The third line contains n integers $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq n$), denoting the colors of the candies.

Output

Print the **maximum sum of the repose** of any subsequence satisfying the condition mentioned.

Example

Input	Output
4 4 3 5 3 2 2 1 1	9



Problem L. What is Uxiono?

Source file name: Uxiono.c, Uxiono.cpp, Uxiono.java, Uxiono.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Jherson Medina - UNAL Bogotá, Colombia

Doctor Padalustro, a renowned interdimensional scientist, is conducting a dangerous experiment with **uxions**, microscopic creatures capable of duplicating themselves at insane speeds.

Padalustro started the experiment by placing **exactly one uxion** in a petri dish. However, uxions have a very peculiar behavior:

- On **odd-numbered minutes**, the number of uxions **grows by the same amount as in the previous minute**.
- On **even-numbered minutes**, the number of uxions **grows by twice the amount that was added in the previous minute**.

The experiment is considered to begin at **minute 0**, at which point there is **exactly 1 uxion**.

Input

The input consists of several test cases.

The first line contains an integer **T** ($1 \leq T \leq 61$), the number of test cases.

Each of the following **T** test cases contains a single integer **M** ($0 \leq M \leq 60$), representing the number of minutes that have passed.

Output

For each test case, print a single integer: the number of uxions after **M** minutes.

Example

Input	Output
4	2
1	4
2	6
3	10
4	

Problem M. Strobogrammatic

Source file name: Strobo.c, Strobo.cpp, Strobo.java, Strobo.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 256 megabytes
Author(s): Roger David Trujillo - UNAL Bogotá, Colombia

An **strobogrammatic** number is a number that reads the same when displayed upside down, that is, when rotated 180 degrees.

Although some people may find these numbers interesting, the truth is that they're highly dependent on the base and the symbols used when represented. In this occasion we'll be focusing only on numbers represented in a hexadecimal base. For the symbols, we will use the standard symbols used in calculators, these ones:



Your goal is to determine what is the minimum of digits that have to be changed for the number to be strobogrammatic.

Input

You're given a non-negative hexadecimal number of at most 10^5 digits consisting only of digits $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, b, C, d, E, F\}$.

Output

Output the minimum number of changes you have to make to the number to make it strobogrammatic.

Example

Input	Output
63181E9	0
4d75b4	5